

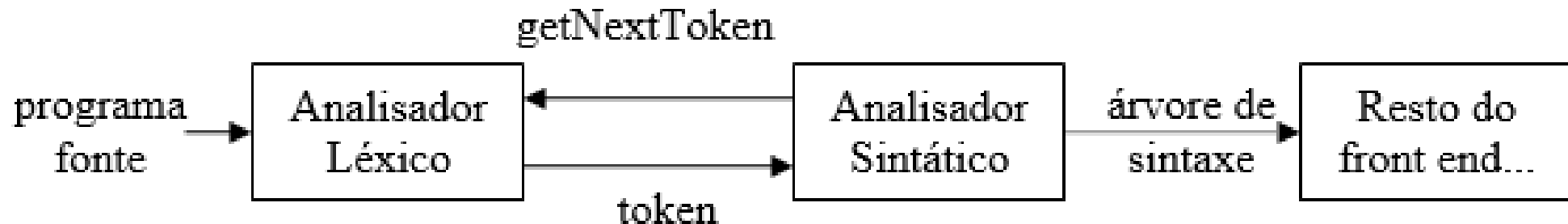
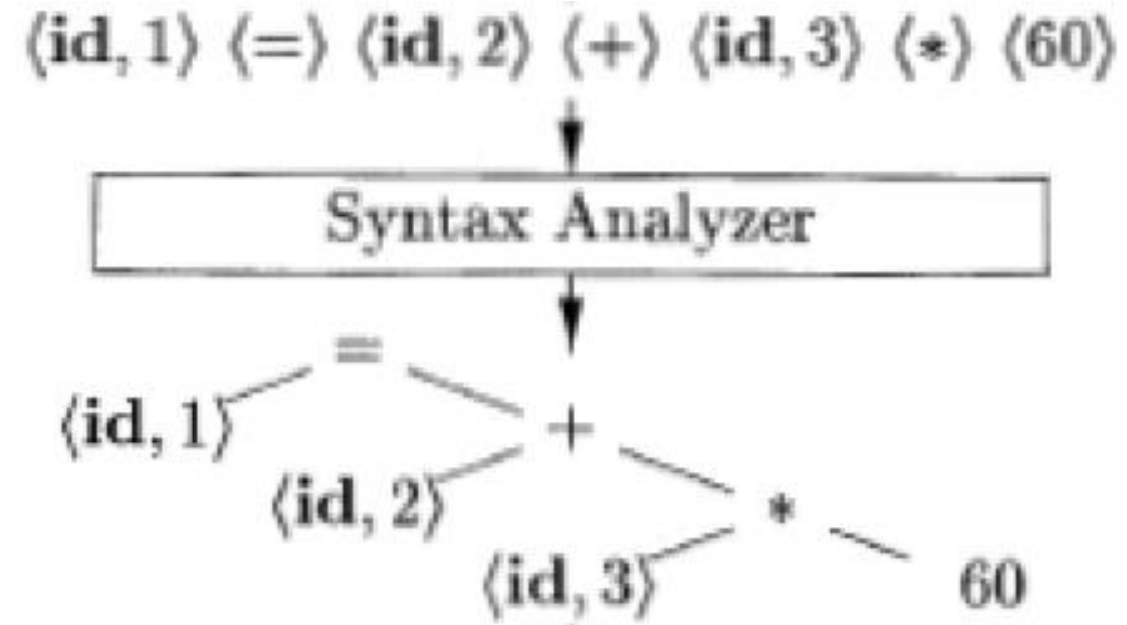
INF 441

Construção de Compiladores

Análise Sintática Top-Down

Analizador Sintático

- obtém tokens do analisador léxico;
- verifica se a sequência de tokens compõe uma forma sentencial válida para a gramática da linguagem, emitindo mensagens de erro, caso necessário;
- constrói uma árvore de sintaxe para o programa de entrada.



Exemplo

Gramática G1:

$$\begin{aligned} E \rightarrow & E + E \mid E - E \\ & \mid E * E \mid E / E \\ & \mid (E) \\ & \mid \text{num} \\ & \mid \text{id} \end{aligned}$$

onde são símbolos terminais: $+ \quad * \quad (\quad) \quad \text{num} \quad \text{id}$

Analizador léxico associa

- num a números inteiros
- id a identificadores

Derivação:

ab + 123

E

$\Rightarrow E + E$

$\Rightarrow \text{id} + E$

$\Rightarrow \text{id} + \text{num}$

(ab é identificado como o terminal id
pelo analisador léxico, e 123 é associado a num)

Exemplo

Gramática G1:

$$\begin{aligned} E \rightarrow & E + E \mid E - E \\ & \mid E * E \mid E / E \\ & \mid (E) \\ & \mid \text{num} \\ & \mid \text{id} \end{aligned}$$

onde são símbolos terminais:

+ - * / () num id

Analizador léxico associa

- num a números inteiros
- id a identificadores

a + b * c

E
=> E + E
=> id + E
=> id + E * E
=> id + id * E
=> id + id * id

Mas G1 é ambígua:

a + b * c

E
=> E * E
=> E + E * E
=> id + E * E
=> id + id * E
=> id + id * id

Gramáticas ambíguas, em geral, não são adequadas para definir o comportamento de analisadores sintáticos.

Exemplo

Gramática G2:

$E \rightarrow E + T \mid E - T \mid T$

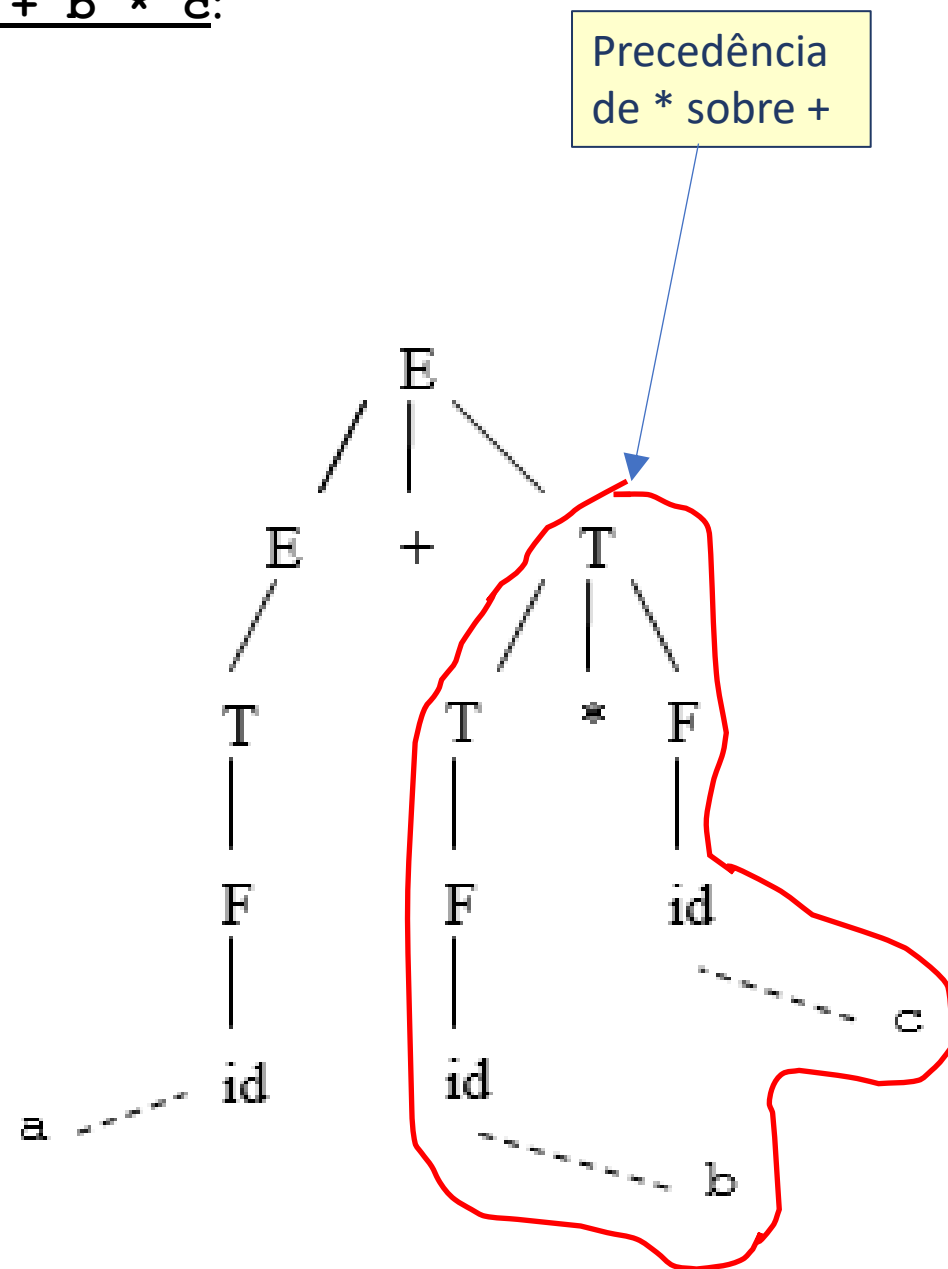
$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{id} \mid \text{num}$

é equivalente a G1, mas não é ambígua.

Única derivação para $a + b * c$:

E
 $\Rightarrow E + T$
 $\Rightarrow T + T$
 $\Rightarrow F + T$
 $\Rightarrow \text{id} + T$
 $\Rightarrow \text{id} + T * F$
 $\Rightarrow \text{id} + F * F$
 $\Rightarrow \text{id} + \text{id} * F$
 $\Rightarrow \text{id} + \text{id} * \text{id}$



Abordagens para construir analisadores sintáticos

- Universal: pode analisar qualquer gramática. Em geral, não são métodos muito eficientes.
- Top-down: constroem árvores sintáticas a partir da raiz para as folhas.
- Bottom-up: constroem árvores sintáticas a partir das folhas até atingir a raiz.

Os analisadores sintáticos top-down e bottom-up mais eficientes funcionam apenas para subclasses de gramáticas livres de contexto. Para atingir eficiência, essas técnicas impõem restrições sobre a forma como as produções podem ser expressas.

Análise Top-Down

- Constrói a árvore de derivação partindo do símbolo inicial da gramática.
- As derivações utilizadas são sempre mais-à-esquerda.
- Em cada passo, a tarefa principal é decidir qual produção a ser aplicada ao terminal mais-à-esquerda.

Exemplo:

Gramática G3:

$S \rightarrow cAd$

$A \rightarrow b \mid a$

Derivação de **cad**:

$S \Rightarrow cAd$ (usando $S \rightarrow cAd$)

$\Rightarrow cbd$ (usando $A \rightarrow b$)

... verifica que não casa com **cad**

$\Rightarrow cAd$ (retorna ao estado anterior)

$\Rightarrow cad$ (usando $A \rightarrow a$)

OK - entrada gerada!

O uso de backtracking não é desejável em analisadores sintáticos por causa do gasto adicional de tempo!

Analísadores Preditivos

- Evitam backtracking.
- A cada passo, analisa os próximos símbolos da entrada (lookahead) para decidir qual produção aplicar.

Gramática G3:

$S \rightarrow cAd$

$A \rightarrow b \mid a$

Próximo símbolo da entrada é "a".

Derivação de **cad**:

Essa informação pode ser usada para decidir qual produção aplicar
($A \rightarrow ???$)

$S \Rightarrow \cancel{c}Ad$ (usando $S \rightarrow cAd$)

$\Rightarrow cad$ (usando $A \rightarrow a$)

A escolha da produção correta é possível analisando apenas 1 símbolo na sequência da entrada (lookahead 1).

Restrições

Gramática G2:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{id} \mid \text{num}$

Derivação para $(((((((((1))))))))))$:

E

$\Rightarrow E + T$

ou

E

$\Rightarrow E - T$

ou

E

$\Rightarrow T$

???

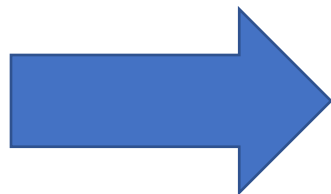
E na derivação para $(((((((((1)))))))))) + 2$???

Nas 3 produções ($E \rightarrow E + T \mid E - T \mid T$), a forma gerada iniciará com “(“. Assim, não se pode decidir qual produção usar, apenas observando um número fixo de símbolos da entrada.

Gramáticas com recursividade à esquerda não são adequadas para produzir analisadores léxicos preditivos com abordagem top-down!

Transformação de gramáticas

Gramática G2:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$


Gramática G4:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid - T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid / F T' \mid \lambda \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$

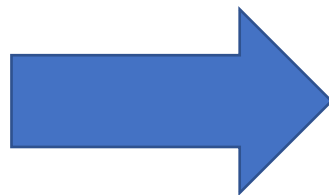
Gramáticas com recursividade à esquerda não são adequadas para produzir analisadores léxicos preditivos com abordagem top-down!

Equivalente a G2, mas sem recursividade à esquerda!

**Perda de legibilidade,
na transformação
de G2 para G4 ...**

Estendendo a notação

Gramática G4':

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid - T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid / F T' \mid \lambda \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$


Gramática G5:

$$\begin{aligned} E &\rightarrow T \ ((+ \mid -) \ T)^* \\ T &\rightarrow F \ ((* \mid /) \ F)^* \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$

ANTLR usa notação similar a essa (maior legibilidade), e aplica análises avançadas para construir analisadores preditivos para casos especiais de gramáticas com recursividade à esquerda.

Construindo analisadores preditivos

- Funções auxiliares:
 - **FIRST**: identifica quais terminais podem aparecer no início de formas sentenciais;
 - **FOLLOW**: identifica quais terminais podem aparecer logo após um símbolo não terminal em formas sentenciais

FIRST

- Para uma palavra w , $\text{FIRST}(w)$ é o conjunto de terminais que podem aparecer no início de palavras derivadas a partir de w .
- Adicionalmente, o símbolo λ pode estar em $\text{FIRST}(w)$, se λ é derivável a partir de w .

G1:

$S \rightarrow ABC \mid D$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

$D \rightarrow d$

Derivando **abc**:

$S \Rightarrow ???$

A produção correta é $S \Rightarrow ABC$ porque “ABC” pode derivar sequência que começa com “a”, enquanto “D” deriva algo que começa com “d”.

FIRST ajuda a decidir:

(aplicando FIRST nos lados direitos das produções)

- $\text{FIRST}(ABC) = \text{FIRST}(A) = \{ a \}$
- $\text{FIRST}(D) = \{ d \}$

FIRST

- Para uma palavra w , $\text{FIRST}(w)$ é o conjunto de terminais que podem aparecer no início de palavras derivadas a partir de w .
- Adicionalmente, o símbolo λ pode estar em $\text{FIRST}(w)$, se λ é derivável a partir de w .

G1:

$S \rightarrow ABC \mid D$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

$D \rightarrow d$

$$\begin{aligned}\text{FIRST}(S) &= \text{FIRST}(ABC) \cup \text{FIRST}(D) \\ &= \{a\} \cup \{d\} = \{a, d\}\end{aligned}$$

Derivando **bac**:

“ERRO: símbolo a ou d esperado”

FOLLOW

- Para não terminal A , $\text{FOLLOW}(A)$ é o conjunto de terminais que podem aparecer imediatamente à direita de A em alguma forma sentencial.
- Pode incluir “\$” (fim da entrada), se A aparecer como símbolo mais à direita de uma forma sentencial.

G2:

$S \rightarrow ABC \mid D$

$A \rightarrow a$

$B \rightarrow b \mid \lambda$

$C \rightarrow c$

$D \rightarrow d$

Derivando **ac**:

S

($S \rightarrow ABC$ porque $\text{FIRST}(ABC)=\{a\}$)
 $\Rightarrow ABC$

($A \rightarrow a$)
 $\Rightarrow aBC$

próximo símbolo da entrada é “c” (**ac**)
usar **$B \rightarrow b$** ou **$B \rightarrow \lambda$** ???

- $\text{FIRST}(b) = \{b\}$
- $\text{FIRST}(\lambda) = \{\lambda\}$

Logo **$B \rightarrow b$** não pode ser usado,
mas e **$B \rightarrow \lambda$** ???

FOLLOW

- Para não terminal A , $\text{FOLLOW}(A)$ é o conjunto de terminais que podem aparecer imediatamente à direita de A em alguma forma sentencial.
- Pode incluir “\$” (fim da entrada), se A aparecer como símbolo mais à direita de uma forma sentencial.

G2:

$S \rightarrow \mathbf{ABC} \mid \mathbf{D}$

$A \rightarrow \mathbf{a}$

$B \rightarrow \mathbf{b} \mid \lambda$

$C \rightarrow \mathbf{c}$

$D \rightarrow \mathbf{d}$

Derivando **ac**:

S

$\Rightarrow ABC$

$\Rightarrow aBC$ (próximo símbolo é “c” em **ac**)
(usar $B \rightarrow \lambda$???)

Após B , aparece um símbolo C em ABC .

$S \rightarrow \mathbf{ABC}$



$\text{FIRST}(C) = \{ c \}$

$\text{FOLLOW}(B) = \text{FIRST}(C) = \{ c \}$

...

$\Rightarrow aBC$

$\Rightarrow aC$

$\Rightarrow ac$

FOLLOW

- Em $S \rightarrow \mathbf{ABC}$:

$$\text{FOLLOW}(\mathbf{B}) = \text{FIRST}(\mathbf{C}) = \{ \mathbf{c} \}$$

- Como calcular FOLLOW para símbolo se ele é o último em lado direito de uma produção?

G3:

$S \rightarrow \mathbf{AE}$

$A \rightarrow \mathbf{CD}$

$C \rightarrow \mathbf{c}$

$D \rightarrow \mathbf{d} \mid \lambda$

$E \rightarrow \mathbf{e}$

Derivando **ce**:

S

$\Rightarrow \mathbf{AE}$

$\Rightarrow \mathbf{CDE}$

$\Rightarrow \mathbf{cDE}$

$\Rightarrow \mathbf{???} \quad \mathbf{D} \rightarrow \mathbf{d} \quad \text{ou} \quad \mathbf{D} \rightarrow \lambda$

Próximo símbolo da entrada é “e” (**ce**)

$\mathbf{D} \rightarrow \mathbf{d}$ não pode ser usado, mas e $\mathbf{D} \rightarrow \lambda$?

FOLLOW

- Em $S \rightarrow \mathbf{ABC}$:
 $\text{FOLLOW}(\mathbf{B}) = \text{FIRST}(\mathbf{C}) = \{ c \}$
- Como calcular FOLLOW para símbolo se ele é o último em lado direito de uma produção?

G3:

$S \rightarrow \mathbf{AE}$

$\mathbf{A} \rightarrow \mathbf{CD}$

$\mathbf{C} \rightarrow c$

$\mathbf{D} \rightarrow d \mid \lambda$

$\mathbf{E} \rightarrow e$

O símbolo “D” só aparece em $\mathbf{A} \rightarrow \mathbf{CD}$ então como saber o que pode vir logo após (FOLLOW) esse símbolo?

Como D é o último símbolo do lado direito de $\mathbf{A} \rightarrow \mathbf{CD}$, então tudo que estiver em $\text{FOLLOW}(\mathbf{A})$ estará em $\text{FOLLOW}(\mathbf{D})$.

$\text{FOLLOW}(\mathbf{A})$

= $\text{FIRST}(\mathbf{E})$ por causa de $S \rightarrow \mathbf{AE}$

= $\{ e \}$

Logo “e” está em $\text{FOLLOW}(\mathbf{D})$.

FOLLOW

- Em $S \rightarrow \mathbf{ABC}$:
 $\text{FOLLOW}(\mathbf{B}) = \text{FIRST}(\mathbf{C}) = \{ c \}$
- Como calcular FOLLOW para símbolo se ele é o último em lado direito de uma produção?

G3:

$S \rightarrow \mathbf{AE}$

$A \rightarrow \mathbf{CD}$

$C \rightarrow \mathbf{c}$

$D \rightarrow \mathbf{d} \mid \lambda$

$E \rightarrow \mathbf{e}$

Derivando **ce**:

S

$\Rightarrow \mathbf{AE}$

$\Rightarrow \mathbf{CDE}$

$\Rightarrow \mathbf{cDE}$

Próximo símbolo da entrada é “e” (**ce**)

$\Rightarrow \mathbf{cE}$ $\mathbf{D} \rightarrow \lambda$ porque “e” $\in \text{FOLLOW}(\mathbf{D})$

$\Rightarrow \mathbf{ce}$

CÁLCULO DE FIRST(x) onde X é qualquer sequência

- se X é apenas um terminal, então $\text{FIRST}(X) = \{ X \}$
- se existe produção $X \rightarrow w$, então $\text{FIRST}(w)$ está incluído em $\text{FIRST}(X)$
- se X é $Y_1 Y_2 \dots Y_n$, e $\lambda \in \text{FIRST}(Y_i)$ para todo $i = 0, \dots, k-1$ ($k < n$), então todo elemento de $\text{FIRST}(Y_k)$ está incluído em $\text{FIRST}(X)$; caso $\lambda \in \text{FIRST}(Y_i)$ para todo $i = 1, \dots, n$, então λ está incluído em $\text{FIRST}(X)$

$$\text{FIRST}(a) = \{ a \}$$

$$A \rightarrow b \mid C$$

$$C \rightarrow d \mid e$$

...

$$\begin{aligned} \text{FIRST}(A) = \\ \{ b \} \cup \text{FIRST}(C) = \\ \{ b, d, e \} \end{aligned}$$

$$A \rightarrow BCD$$

$$B \rightarrow b \mid \lambda$$

$$C \rightarrow c \mid \lambda$$

$$D \rightarrow d$$

...

$$\begin{aligned} \text{FIRST}(A) = \\ \{ b, c, d \} \end{aligned}$$

CÁLCULO DE FOLLOW(A) onde A é símbolo não terminal

- se A é o símbolo inicial da gramática, então FOLLOW(A) contém \$ (“fim de arquivo”)
- se existe produção $B \rightarrow uAv$, então todos os elementos de FIRST(v), exceto λ , estão incluídos em FOLLOW(A)
- se existe produção $B \rightarrow uA$ ou uma produção $B \rightarrow uAv$ onde FIRST(v) contém λ , então todos os elementos de FOLLOW(B) estão incluídos em FOLLOW(A).

S $\rightarrow$...

FOLLOW(S) =
{ \$, ... }

B $\rightarrow$ **aAC**

C $\rightarrow$ **d | e | λ**

...

FOLLOW(A) =
{ d, e, ... }

B $\rightarrow$ **aA**

C $\rightarrow$ **Bc | Bd**

...

FOLLOW(A) =
{ c, d, ... }

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E'$
 $\quad \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T'$
 $\quad \mid \lambda$
 $F \rightarrow (E)$
 $\quad \mid id$
 $\quad \mid num$

$FIRST(E)$
 $= FIRST(T)$
 $= FIRST(F)$
 $= \{ (, id, num \}$

$FIRST(E')$
 $= \{ +, \lambda \}$

$FIRST(T')$
 $= \{ *, \lambda \}$

$FOLLOW(E) =$
 $\{ \$,) \}$

$FOLLOW(E')$
 $= FOLLOW(E)$
 $= \{ \$,) \}$

$FOLLOW(T)$
 $= FIRST(E')$
 $\quad \cup FOLLOW(E')$
 $= \{ +, \$,) \}$

$FOLLOW(T')$
 $= FOLLOW(T)$
 $= \{ +, \$,) \}$

$FOLLOW(F)$
 $= FIRST(T')$
 $\quad \cup FOLLOW(T)$
 $\quad \cup FOLLOW(T')$
 $= \{ *, +, \$,) \}$

Gramáticas LLK

Para a classe de gramáticas chamada LL(k), sempre é possível construir analisadores sintáticos preditivos.

L : Left to right

(leitura da entrada da esquerda para a direita)

L : Leftmost derivation

(derivação mais-à-esquerda)

k : para decidir qual produção usar, k símbolos da entrada são usados como lookahead

Tabela para gramática LL(1)

A tabela é uma matriz M de 2 dimensões:

- uma linha para cada símbolo não terminal da gramática
- uma coluna para cada símbolo terminal da gramática (incluindo símbolo $\$$, que representa fim da entrada).

Para cada produção $A \rightarrow u$:

1. Para cada terminal a em $\text{FIRST}(u)$, adicione $A \rightarrow u$ a $M[A,a]$.
2. Se λ pertence a $\text{FIRST}(u)$ então para cada símbolo b em $\text{FOLLOW}(A)$, adicione $A \rightarrow u$ a $M[A,b]$. O símbolo b pode ser inclusive $\$$.

Exemplo

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \lambda \\ F &\rightarrow (E) \mid id \end{aligned}$$

$$\begin{aligned} FIRST(T E') &= \{ (, id \} \\ FIRST(F T') &= \{ (, id \} \\ FOLLOW(E) &= \{), \$ \} \\ FOLLOW(E') &= \{), \$ \} \\ FOLLOW(T) &= \{ +,), \$ \} \\ FOLLOW(T') &= \{ +,), \$ \} \\ FOLLOW(F) &= \{ +, *,), \$ \} \end{aligned}$$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E						
E'						
T						
T'						
F						

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \lambda$
 $F \rightarrow (E) \mid id$

$FIRST(T E') = \{ (, id \}$
 $FIRST(F T') = \{ (, id \}$

$FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +,), \$ \}$
 $FOLLOW(T') = \{ +,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	E → T E'			E → T E'		
E'						
T						
T'						
F						

Exemplo

$$\mathbf{E} \rightarrow \mathbf{T} \mathbf{E}'$$
$$\mathbf{E}' \rightarrow + \mathbf{T} \mathbf{E}' \mid \lambda$$

T $\rightarrow$ **F** **T** \

$$T' \rightarrow * F T' \mid \lambda$$
$$\mathbf{F} \rightarrow (\mathbf{E}) \mid \text{id}$$
$$\text{FIRST}(T \rightarrow E') = \{ (\epsilon, \text{id}) \}$$
$$\text{FIRST}(F \rightarrow T') = \{ (\epsilon, \text{id}) \}$$
$$\text{FOLLOW}(\text{E}) = \{) , \$ \}$$
$$\text{FOLLOW}(E') = \{ \text{) } , \$ \}$$
$$\text{FOLLOW}(\text{T}) = \{ +,), \$ \}$$
$$\text{FOLLOW}(\mathbf{T}') = \{ +,), \$ \}$$
$$\text{FOLLOW}(\mathbb{F}) = \{ +, *,), \$ \}$$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$ $E' \rightarrow + T E'$			$E \rightarrow T E'$		
E'						
T						
T'						
F						

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \lambda$
 $F \rightarrow (E) \mid id$

$FIRST(T E') = \{ (, id \}$
 $FIRST(F T') = \{ (, id \}$

$FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +,), \$ \}$
 $FOLLOW(T') = \{ +,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	E → T E'	E' → + T E'		E → T E'		
E'						
T				T → F T'		
T'						
F						

Exemplo

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \lambda \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \lambda \\ F &\rightarrow (E) \mid id \end{aligned}$$

$$\begin{aligned} \text{FIRST}(T E') &= \{ (, id \} \\ \text{FIRST}(F T') &= \{ (, id \} \\ \text{FOLLOW}(E) &= \{), \$ \} \\ \text{FOLLOW}(E') &= \{), \$ \} \\ \text{FOLLOW}(T) &= \{ +,), \$ \} \\ \text{FOLLOW}(T') &= \{ +,), \$ \} \\ \text{FOLLOW}(F) &= \{ +, *,), \$ \} \end{aligned}$$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	E → T E' T → F T'	E' → + T E'	T' → * F T'	E → T E'		
E'						
T				T → F T'		
T'						
F						

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \lambda$
 $F \rightarrow (E) \mid id$

$FIRST(T E') = \{ (, id \}$
 $FIRST(F T') = \{ (, id \}$
 $FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +,), \$ \}$
 $FOLLOW(T') = \{ +,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	E → T E'	E' → + T E'		E → T E'		
E'						
T	T → F T'			T → F T'		
T'				T' → * F T'		
F	F → id			F → (E)		

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \lambda$
 $F \rightarrow (E) \mid id$

$FIRST(T E') = \{ (, id \}$
 $FIRST(F T') = \{ (, id \}$
 $FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +,), \$ \}$
 $FOLLOW(T') = \{ +,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$	$E' \rightarrow + T E'$		$E \rightarrow T E'$	$E' \rightarrow \lambda$	$E' \rightarrow \lambda$
E'						
T	$T \rightarrow F T'$		$T' \rightarrow * F T'$	$T \rightarrow F T'$		
T'						
F	$F \rightarrow id$			$F \rightarrow (E)$		

Exemplo

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \lambda$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \lambda$
 $F \rightarrow (E) \mid id$

$FIRST(T E') = \{ (, id \}$
 $FIRST(F T') = \{ (, id \}$
 $FOLLOW(E) = \{), \$ \}$
 $FOLLOW(E') = \{), \$ \}$
 $FOLLOW(T) = \{ +,), \$ \}$
 $FOLLOW(T') = \{ +,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$	$E' \rightarrow + T E'$		$E \rightarrow T E'$	$E' \rightarrow \lambda$	$E' \rightarrow \lambda$
E'						
T	$T \rightarrow F T'$			$T \rightarrow F T'$		
T'						
F	$F \rightarrow id$			$F \rightarrow (E)$		

Analizador preditivo não recursivo

Uma pilha armazena os símbolos ainda não utilizados das formas sentenciais.

A tabela M construída para o analisador preditivo é usada da seguinte forma:

- Se no topo da pilha houver um símbolo não terminal, a tabela é consultada usando esse símbolo e o próximo símbolo terminal da entrada. O não terminal é substituído pelo lado direito da produção identificada.
- Se houver terminal no topo da pilha, ele deve “casar” com o próximo símbolo terminal da entrada.

Exemplo: **id+id*id**

entrada casada	pilha	entrada restante	ação
		)	$E \rightarrow T E'$
	T E' \$	id + id * id \$	$T \rightarrow F T'$
	F T' E' \$	id + id * id \$	$F \rightarrow id$
	id T' E' \$	id + id * id \$	casa com id
id	T' E' \$	+ id * id \$	$T' \rightarrow \lambda$
id	E' \$	+ id * id \$	$E' \rightarrow + T E'$
id	+ T E' \$	+ id * id \$	casa com +
id +	T E' \$	id * id \$	$T \rightarrow F T'$

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$	$E' \rightarrow + T E'$		$E \rightarrow T E'$	$E' \rightarrow \lambda$	$E' \rightarrow \lambda$
E'						
T	$T \rightarrow F T'$	$T' \rightarrow \lambda$	$T' \rightarrow * F T'$	$T \rightarrow F T'$	$T' \rightarrow \lambda$	$T' \rightarrow \lambda$
T'						
F	$F \rightarrow id$			$F \rightarrow (E)$		

não terminal	símbolo de entrada					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$ $T \rightarrow F T'$ $F \rightarrow id$	$E' \rightarrow + T E'$ $T' \rightarrow \lambda$	$T' \rightarrow * F T'$	$E \rightarrow T E'$	$E' \rightarrow \lambda$ $T' \rightarrow \lambda$	$E' \rightarrow \lambda$ $T' \rightarrow \lambda$
E'						
T				$T \rightarrow F T'$		
T'						
F				$F \rightarrow (E)$		

entrada casada	pilha	entrada restante	ação
id +	F T' E' \$	id * id \$	F → id
id +	id T' E' \$	id * id \$	casa com id
id + id	T' E' \$	* id \$	T' → * F T'
id + id	* F T' E' \$	* id \$	casa com *
id + id *	F T' E' \$	id \$	F → id
id + id * id	id T' E' \$	id \$	casa com id
id + id * id	T' E' \$	\$	T' → λ
id + id * id	E' \$	\$	E' → λ
			Aceita!